Part Three

Tyler Dinh[1], Kevin Tang[1], and Demi Babalola[1]

[1]Washington University in St. Louis

October 14, 2026

1 Algorithmic Design and Implementation Details

The objective function is Mean Squared Error (MSE), defined as $MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$.

Then, the gradient equation used for parameter updates for step i can be calculated as:

$$w^{i+1} = w^i - \alpha\frac{\delta MSE}{\delta m}$$

$$\frac{\delta MSE}{\delta m} = \frac{-2}{n}\sum_{i=1}^{n}(y_i - (mx_i + b))$$

Algorithm 1 Custom Gradient Descent Function

procedure GRADIENT DESCENT(W^0, b^0, X, y, η, N)
  for $i = 0$ to N **do**
    $\text{err} = f(X, W^i, b^i) - y$
    $W^{i+1} = W^i - \eta \cdot \frac{2}{|X|}X^\top \text{err}$
    $b^{i+1} = b^i - \eta \cdot 2\,\overline{\text{err}}$
  end for
  return W^N, b^N
end procedure

Our dependencies were:

ucimlrepo 0.0.7, pandas 3.0.5, matplotlib 3.11.2, numpy 2.5.3, scikit-learn 1.9.1, statsmodels 0.15.0, python 3.12

We designated rows 501–630 as the testing set (130 samples) and the remaining 900 samples as the training set, and split into a features matrix and output vector for each set. Model parameters were not updated based on the test dataset.

We set a list of learning rates for α = [1e-9, 1e-8, 1e-7], initial coefficient weights matrix to be all zeroes, initial bias to be zero, and iterations to be 1000. We stop the algorithm after i=1000 completes.

Then, loop over lrs and find the coefficient weights matrix, bias, along with mse and r2 over time as requested.

2 Feature Scaling and Gradient Descent Dynamics

Chart here from assets/unscaled vs scaled

Feature scaling (normalization by z-score) allowed the weights of the model to converge more quickly. Because the values in general are smaller than without scaling, we tested a higher-valued set of learning rates. When we used the smaller learning rates we had to use for unnormalized data, the model did not learn at a sufficient pace or converge. Also, scaling brings the values from different features closer to each other, so training using scaled features becomes smoother as well, as shown in the above chart. Overall, we find that our algorithm performs better on the test set (lower MSE and higher R2) when we scale the features.

A linear transformation does not meaningfully change the optimal mathematical solution. Although the weights are different for a scaled model, the output of the scaled model on scaled inputs is identical to the output of the unscaled model on unscaled inputs. We can see that the MSE on the test set for both a scaled and unscaled model are identical, at 141.25799330049728 and 0.35125143823506555 respectively.

3 Linear vs. Non-Linear Feature Transformations

chart here from assets/nonlinear

The log transformation increases convergence speed and stability greatly over the unscaled features, and we were able to use much higher learning rates without the model diverging. This is expected, because similarly to the z-score scaling, it brings the distribution of all the features closer to one another by bringing down extremely large values so that gradient descent proceeds more smoothly.

table here from assets/nonlinear table

While the untransformed and normalized models perform the same on the testing and training sets, the log transform, which is nonlinear, improved the optimal solution and overall predictive performance. This is because the log transformation helps with features that don't have a linear relationship with the target feature, as well as making right-skewed distributions more normal by stretching out low-valued data and compressing high-valued data, which reduces heteroscedasticity.

4 Univariate vs. Multivariate Model Dyanamics

5 Statistical Inference and Regression Analysis

6 Code and Reproducibility Access